

Manual tecnico del codigo

Compilador Semantico Web

Este documento explica la estructura interna del proyecto, la funcion de cada archivo y las zonas que se deben modificar si se desea ampliar el analizador.

Campo	Detalle
Proyecto	Compilador Semantico Web
Integrantes	Gabriel Argenis Medina Carrero - Kevin Garcia
Curso	Teoria de Compiladores
Docente evaluador	Alvaro Salamanca
Version documentada	v1.5.7 comentada

1. Arquitectura general

El proyecto es una aplicacion web estatica. No usa base de datos ni backend. La interfaz esta en HTML y CSS, mientras que la logica del compilador se ejecuta en el navegador por medio de JavaScript. Para despliegue se usa Nginx dentro de Docker.

2. Flujo de ejecucion

El usuario escribe codigo en el editor. Al presionar Analizar codigo, analyzer.js toma el texto, lo tokeniza, valida reglas sintacticas, construye simbolos, revisa tipos y finalmente muestra resultados en las pestanas de la interfaz.

3. index.html

Define la estructura visual: encabezado, secciones informativas, demo interactiva, pestanas de resultados, documentacion, seccion Acerca de y footer. Los elementos con id como codeInput, runButton, errors, symbols, tokens e intermediate son usados desde JavaScript.

4. styles.css

Controla colores, margenes, tarjetas, editor, botones, pestanas, responsividad y la identidad visual. Las variables globales se encuentran en :root. Si se desea cambiar el color principal, debe modificarse --usb-green o las variables relacionadas.

5. analyzer.js

Es el archivo mas importante. Contiene el motor del analizador: ejemplos, tokens, validaciones, tabla de simbolos, evaluacion de expresiones y renderizado de resultados.

6. Dockerfile y docker-compose.yml

Dockerfile crea una imagen con Nginx y copia el proyecto a /usr/share/nginx/html. docker-compose.yml publica el puerto 80 del contenedor en el puerto 8097 del servidor.

7. Funciones principales de analyzer.js

Funcion o bloque	Responsabilidad
phaseInfo	Textos usados para explicar las fases del compilador.
examples	Codigos precargados que se cargan en el editor.
tokenizeLine()	Divide una linea en tokens.
tokenize()	Tokeniza todo el programa.
analyze()	Funcion central que revisa el programa completo.
issue()	Crea objetos de error con linea, tipo y sugerencia.
findSymbol()	Busca variables declaradas y visibles.
findSimilarSymbol()	Sugiere nombres parecidos cuando hay errores de escritura.
evaluateExpression()	Infiere el tipo de una expresion.
renderAnalysis()	Actualiza la interfaz con todos los resultados.
renderErrors(), renderSymbols(), renderTokens()	Actualiza la tabla de simbolos y tokens.

8. Que modificar segun la necesidad

Necesidad	Archivo / zona recomendada
Cambiar colores o estilo visual	styles.css - variables en :root y clases de secciones.
Agregar un tipo de dato	analyzer.js - TYPE_KEYWORDS, suggestedTypeFor(), assignmentProblem().
Agregar una instruccion nueva	analyzer.js - analyze() y evaluateExpression() si aplica.
Cambiar mensajes de error	analyzer.js - issue(), undeclaredIssue() y validaciones dentro de analyze().
Cambiar textos de fases	analyzer.js - objeto phaseInfo.
Cambiar ejemplos	analyzer.js - objeto examples.
Cambiar informacion academica	index.html - seccion Acerca de.
Cambiar puerto local	docker-compose.yml - ports.

9. Ejemplo de flujo interno

Usuario escribe:
entero edad = 20;
edad = edad + 1;

Flujo:
1. codeInput obtiene el texto.
2. tokenize() separa tokens.
3. analyze() detecta declaracion y asignacion.
4. La tabla de simbolos registra edad como entero.
5. renderAnalysis() muestra resultados en la interfaz.

10. Recomendaciones de mantenimiento

- No eliminar la linea script src='analyzer.js', porque los botones dejan de funcionar.
- Si se cambia styles.css o analyzer.js, aumentar el parametro ?v para evitar cache del navegador.
- Mantener index.html, styles.css y analyzer.js separados para que sea facil de revisar.

- No subir archivos .env, tokens, claves ni carpetas .git en entregas academicas.
- Si el analizador crece mucho, separar la logica en modulos o moverla a un backend.